

2400-DS1AL # Algorithms for Data Science

[Kokpit](#) / [Moje kursy](#) / [2400-DS1AL#2025L#277553](#) / [Lecture 12 & 13](#) / [Lecture notes](#)

Lecture notes

Credits: dr Krzysztof Fleszar

Lecture 8: Graphs

A graph is a set of *vertices* connected with *edges*. Every edge connects two vertices; in *undirected* graphs the connection is two-way, while in *directed* graphs the connection is from the source vertex to the target vertex. We also consider graphs where the edges have additional associated values (usually called *weights* or *costs*); in the algorithms presented here we always assume that the costs are greater than 0. We usually use n to denote the number of vertices and m to denote the number of edges; we use these conventions when giving the computational complexity of our algorithms.

All the algorithms discussed below work also for graphs with *multiple* edges (also called *parallel* edges); these are edges that connect the same pair of vertices (in the same direction in the case of directed graphs).

Why are Graphs Useful?

In short, there are two important usages. First, many problems can be modeled as graph problems and then solved using known graph algorithms. Second, data can be often represented by graphs. Using adequate graph visualization techniques, one might infer new insights from the data by just "looking" at such graph drawings. In this course, we will focus on the first usage.

For a data scientist, probably the most natural examples of graphs to consider are social networks. The vertices are people and the edges describe their relationships. For instance, we could consider a graph that has an edge between a person A and a person B if they are friends on Facebook (undirected graph), or we have an edge from A to B if A follows B on Twitter (directed graph).

Road networks are another important example. We would probably model these as weighted graphs where an edge corresponds to a road segment. The weight of an edge could be, e.g., the time necessary to travel through the respective road segment.

Graphs are also an extremely useful abstract notion. Many computational problems can be stated in terms of graphs, and solved using graph algorithms. For example, take a look at the following video:

It shows an algorithm playing Manic Miner, and solving the levels optimally, i.e., in the shortest amount of time. This is done by modelling every possible state of the game (how much time has passed, where is Miner Willy at the moment, which items has he already collected) as a vertex of a graph. For each state, we consider all the possible keyboard keys that the player can press, and create edges to each state obtained from such a keypress. Solving the level optimally then boils down to finding the shortest path in this graph.

How to Represent Graphs?

An edge from a vertex a to a vertex b is just a tuple (a,b) . If it has some cost c , is its a triple (a,b,c) . Thus, mathematically, a graph can be fully described by the (multi) set of tuples/triples representing the edges (together with the set of vertices for the case that we have vertices without edges). However, such a representation is not always efficient inside a program.

For graph algorithms, there are two better ways to represent a graph (click here for a [visualization](#)):

- **Adjacency matrix**—this is an $n \times n$ matrix such that $\text{adjacency_matrix}[i][j]$ is 1 if there is an edge from the i -th vertex to the j -th vertex, and 0 otherwise. The edge weights can be stored in an analogous weight matrix. *(This representation works also for unweighted graphs with parallel edges: in $\text{adjacency_matrix}[i][j]$ we just store the number of edges from i to j . However, the representation becomes impractical for weighted graphs with parallel edges.)*
- **Adjacency lists**—for every vertex, we have a list of edges starting in this vertex. Each such edge can be represented just by the target vertex and its weight, for instance, as tuples. *(Naturally, this representation works also for weighted graphs with parallel edges.)*

In practice, adjacency lists are often more efficient with respect to time and memory: adjacency lists require only linear $\Theta(n+m)$ space whereas adjacency matrices require quadratic $\Theta(n^2)$ space. Thus, just reading an adjacency matrix into the memory takes already $\Omega(n^2)$ time. Furthermore, the same quadratic time is needed to go through all the edges, which is much more than the $O(n+m)$ time for the case of adjacency lists. However, depending on the application and the graph, adjacency matrices can be better: with an adjacency matrix, we can check the existence of a single edge in constant time, whereas with adjacency lists it can take up to $\Omega(m)$ time. Also note that the advantages of adjacency lists become less significant if the graph has a lot of edges, that is, if $m=\Omega(n^2)$.

Shortest/Cheapest Path Algorithms

A *path* in a graph from a vertex s to a vertex t is a sequence of vertices $v_0, \dots, v_\ell$ such that $v_0=s$, $v_\ell=t$, and there is an edge from v_i to v_{i+1} . The length of a path is the number of its edges (thus ℓ for the path $v_0, \dots, v_\ell$), whereas the cost/weight of a path is the total weight of its edges.

In unweighted graphs, we are usually interested in the shortest path, that is, one of smallest possible length (hence, smallest number of edges). In weighted graphs, we want to know the cheapest (lightest) path, that is, a path of smallest possible cost (hence, smallest total weight of the edges).

In social network analysis, the length of a shortest path tells us how distant two people are; in a road network, the cost of a lightest path tells us how to go from s to t in the cheapest possible way.

In the following, we take a look at several algorithms that compute a shortest or lightest path. All those algorithms work as well for undirected as for directed graphs.

Take a look at this [visualization](#) to see how the algorithms below (BFS, DFS, Dijkstra and Floyd-Warshall) work step-by-step.

If you want to see actual implementations of the algorithms (BFS, DFS, Dijkstra and Floyd-Warshall), then click here: [code](#) in Python, [code](#) in C++.

Unweighted Graphs: Breadth First Search (BFS)

Given a source vertex s , we will compute a shortest path from s to any vertex w . For unweighted graphs, we can do this with a very simple algorithm, called breadth first search (BFS). The idea of BFS is that first we find all the vertices in distance 1 to s , then we try their outgoing edges to find all the vertices in distance 2 to s , then we try their outgoing edges to find all the vertices in distance 3 to s , and so on. To keep track which vertices should be visited next, we use the data structure called *queue*: we can insert objects to the queue, and later remove them; the first item added is the first item removed, the second item added is the second item removed, and so on. Hence, it is a "first-in-first-out" data structure and its add/remove operations take $O(1)$ time.

The Queue Data Structure:

- `add(v)`: inserts an element with value v into the queue. (Alternative name: `enqueue(v)`)
- `findOldest()`: returns the value of the oldest element. (Alternative name: `peek()`)
- `extractOldest()`: removes the oldest element from the queue and returns its value. (Alternative name: `dequeue()`)

All operations in $O(1)$ time.

Intuition: We add at the end, and remove from the beginning. Hence, all elements have to wait in a queue.

In pseudocode, the BFS algorithm works as follows:

- We have a queue q and an array $dist[1..n]$ of distances that we have already computed, where $dist[i]$ is the distance of vertex i from s (here we assume that the vertices are numbered from 1 to n ; in Python/C++ we would number them from 0 to $n-1$).
- Insert s in q , and set $dist[s]=0$.
- While q is not empty:
 - Remove the first vertex v from q (which is the vertex that has been added first among the vertices currently in the queue).
 - For every edge (v,w) , i.e., from v to w : if we do not know $dist[w]$ yet, we set $dist[w]=dist[v]+1$, and we insert w to q .

The time complexity is $\Theta(n+m)$, and the memory complexity is $\Theta(n)$.

The solution above computes only the lengths of the shortest paths. To compute the actual paths, it suffices to have another table $parents[1..n]$ and, each time, when we compute $dist[w]=dist[v]+1$, we also set $parent[w]=v$. In this way, the shortest path from s to v can be obtained by walking backwards from v to s :

$v, parent[v], parent[parent[v]], \dots, s$.

Unweighted Graphs: Depth First Search (DFS)

A similar algorithm to BFS is depth first search (DFS), with essentially the only difference that we use a *stack* instead of a *queue* (there are also some minor other differences). A stack is a "last-in-first-out" structure: when we remove an element, we remove the element that has been added most recently.

The Stack Data Structure:

- `add(v)`: inserts an element with value v into the stack. (Alternative name: `push(v)`)
- `findNewest()`: returns the value of the newest element. (Alternative name: `top()`)

- `extractNewest()`: removes the newest element from the stack and returns its value. (Alternative name: `pop()`)

All operations in $O(1)$ time.

Intuition: We add elements on top of a pile (stack) and remove them also from the top.

Since we use a stack instead of a queue, a DFS search looks as follows: we go along any edge starting at s and reach some vertex u , then we follow any edge from u to reach another (yet unvisited) vertex v , then we follow any edge from v to some unvisited vertex and so on. Thus, initially, the DFS search follows some arbitrary path until it reaches some vertex w whose all neighboring vertices have already been visited. (Hence the name: in DFS we first go as "deep" as possible into the graph whereas in BFS we first visit all neighbors around s .) Then, if all neighbors of the last vertex w have been visited, observe that guided by the nature of the stack operations, DFS goes back along the search path and checks whether there are still unvisited neighbors from the preceding vertices that it might follow. Time and memory complexity of DFS are the same as the ones of BFS since the add/remove operations of a stack take also constant time.

Both, BFS and DFS, can be used to check which pairs of vertices are connected (by any path). However, the paths computed by **DFS** (which we can obtain by using, e.g., the *parent* table as in BFS) are **not** necessarily **shortest paths**. Instead, DFS computes a visit order of the vertices, also called topological order, and has its specific uses in more complex graph algorithms. In some situations, DFS might be easier to implement than BFS because a stack is a simpler structure than a queue that can be even replaced by using recursion.

Weighted Graphs: Dijkstra's Algorithm

Now, consider a weighted graph, where we want to compute the cheapest path from a given s to any vertex w . BFS will not compute them correctly—it finds the *shortest* path, which is not necessarily the cheapest one. We could fix BFS by inserting a vertex v to the queue again when we have found a path to v which is cheaper than what we already had; however, this method yields a potentially much less efficient algorithm (it is possible that we will have to insert every vertex many times—even exponentially many times).

Instead, we have to adjust the idea of BFS such that we still visit a vertex only once. BFS worked because we considered all the vertices in a specific order: in the order of their distance to s measured by the length of a shortest path. We need to do the same, but this time considering the lightest paths instead of simply the number of edges. Thus, instead of a simple queue that removes the first element that has been added (hence, an element that has the smallest distance measured by the shortest path length), we need a more advanced data structure that removes an element with the smallest distance measured with respect to the lightest path. In other words, we need a so-called *priority queue* that removes an element with the highest priority where the priority is the higher, the smaller the total weight of the lightest path is.

The **Priority Queue** Data Structure:

- $\text{add}(v, p)$: inserts an element with value v and priority p into the priority queue. Time: $O(\log m)$ where m is the number of elements in the data structure.
- $\text{findMax}()$: returns the value of the element with the highest priority. Time: $O(1)$.
- $\text{extractMax}()$: removes the element with the highest priority from the priority queue and returns its value. Time: $O(\log m)$ where m is the number of elements in the data structure.

Dijkstra's algorithm works as follows:

- We have a priority queue q , and again an array $\text{dist}[1..n]$ of distances that we have already computed (this time not measured by the number of edges but by the total edge weight of a lightest path),
- Insert s in q , and set $\text{dist}[s]=0$.
- While q is not empty:
 - Remove a vertex v from q that has the smallest distance $\text{dist}[v]$.
 - If v was removed from q the first time:
 - For every edge (v, w, c) , i.e., from v to w with cost c : if we do not know $\text{dist}[w]$ yet, or if $\text{dist}[w] > \text{dist}[v] + c$, we set $\text{dist}[w] = \text{dist}[v] + c$, and we insert w to q .

Why do we check whether v has been removed from q the first time? Observe that it might happen that a vertex v is added several times to the queue (in the worst case for every edge leading to v). This means that v will be removed several times from q . However, one can show that the first time that we remove v from q , we can be sure that $\text{dist}[v]$ is already optimal. Consequently, subsequent removals of v will not change the dist -values of its neighbors. Therefore, to keep the running time low, we only go through the neighbors of v if v has been removed for the first time.

And how to find out whether a vertex v has just been removed for the first time from q ? We just maintain another array $\text{removed}[1..n]$ initially set to false. After removing v for the first time, we note this fact by setting $\text{removed}[v]=\text{true}$.

To obtain the actual cheapest paths, we can use a *parent* table as we did in BFS.

The question remains how to implement a priority queue, that is, how to find v for which $\text{dist}[v]$ is the smallest. It turns out that the *heap* data structure, which we have used in heap sort (Lecture 5: Sorting), is perfect for this. To add a vertex v to q , we just insert the pair $(-\text{dist}[v], v)$ to the heap and keep the heap invariant with respect to the key. Note that the element in the root is always the one with the greatest key, i.e., with the greatest $-\text{dist}[v]$ value, i.e., with the smallest $\text{dist}[v]$ value. Since we add not more elements to the heap than we have edges, the number of elements

in the heap will never exceed m . Thus, the running time of each add or remove operation is bounded by $\Theta(\log m)$ (recall Lecture 5). Naturally, the number of remove operations is bounded by the number of add operations which again is bounded by m . Thus, the total running time is $\Theta(n+m\log m)$.

As noted above, a single vertex might be added several times to the heap. One could do a more careful implementation of a priority queue and instead of re-adding a vertex, one could just update its key and move it along the heap to its new position using the upheap function from Lecture 5 (we could, for instance, use a hash table to find the position of the vertex within the heap or just use a balanced search tree for the whole data structure). Such an implementation reduces the running time to $\Theta(n+m\log n)$; however, in general this is not worth the extra effort—first, usually we do not have parallel edges, and thus $m < n^2$ implying that there is no difference between $\Theta(\log n)$ and $\Theta(\log m)$; second, the implementation is much more complex (in particular, common implementations of heaps such as `priority_queue` in C++ and `heapq` in Python do not allow to change keys easily).

Additional note: the time complexity of Dijkstra's algorithm can be reduced to $\Theta(m+n\log n)$ by using so-called *Fibonacci heaps* instead of common binary heaps.

From Everywhere to Everywhere: Floyd-Warshall Algorithm

In the Floyd-Warshall algorithm, we compute a cheapest path from any s to any t .

The example input in the Python code below corresponds to the following graph. Note that the graph contains two multiple edges for the pair $(0,1)$. Normally, we assume that a graph has at most one edge per vertex pair.

 The graph corresponds to the example input in the Python code.

```
# Number of vertices
n = 5

# It is sufficient to have the edges as a single big list (source, target, cost)
edges = [(0, 1, 10), (0, 1, 8), (1, 2, 15), (0, 2, 22), (2, 3, 40), (3, 4, 8), (4,0, 100)]

# At all times, delta[i][j] will be the upper bound on the cost of the
# cheapest path from i to j.

# Infinity: Used to signalize that we haven't found any path for
# the respective pair yet. Actually, we use a very large number
# (one that is surely greater than the cost of any actual cheapest path)
INF = 10000000000

# Initially, we do not know anything about any paths...
delta = [ [INF] * n for v in range(n) ]

# We know that for any edge (s,t,c), a cheapest path from s to t is upper bounded by c.
# We use min to deal with the case of multiple edges between the same pair of vertices.
for (s,t,c) in edges:
    delta[s][t] = min(delta[s][t], c)

# We also know that the distance from a vertex to itself is always 0
for v in range(n):
    delta[v][v] = 0

for k in range(n):
    for i in range(n):
        for j in range(n):
            # If delta[i][k] + delta[k][j] (the cost of the best known path from i to j
            # going through k)
            # is smaller than delta[i][j] (the cost of the best known path from i to j),
            # replace delta[i][j]
            delta[i][j] = min(delta[i][j], delta[i][k] + delta[k][j])

# At then end, for every (s,t),
```



```
# delta[s][t] will be the cost of the cheapest path from s to t,
# or INF if no s-t-path exists at all.
```

The order of the k, i, j loops is important. The outermost loop has the following invariant: for each i, j , $\text{delta}[i][j]$ is the cost of the cheapest path from i to j , among the paths which only go through vertices $0, \dots, k-1$ —hence, the vertices that have previously been considered as k on the way. (This algorithm is often considered a dynamic programming algorithm—after each k we have a partial solution.) The invariant is helpful as follows: if the invariant holds, then we know that at the end, when the outermost loop ends, $\text{delta}[i][j]$ is the cost of the cheapest path from i to j without any restrictions.

So how to show that the invariant holds? In the beginning, this invariant holds when no k was considered yet as we have already computed all direct paths (i.e., consisting of a single edge). It is sufficient to prove that, if the invariant holds before any one iteration, then it also holds after that iteration: the cheapest path from i to j which could only go through vertices $0, 1, \dots, k$ either does not actually use k (then $\text{delta}[i][j]$ already knew this path before the iteration k), or it does, which means that it is a combination of some path from i to k going through only $0, \dots, k-1$, and some path from k to j going through only $0, \dots, k-1$ —the cost of the cheapest such path is $\text{delta}[i][k] + \text{delta}[k][j]$, so our algorithm will find it.

The Floyd-Warshall algorithm runs in time $O(n^3 + m)$ and memory $O(n^2)$. We could also use the Dijkstra algorithm n times—the running time of a simple implementation is $O(n^2 + nm \log n)$. Dijkstra algorithm wins for sparse graphs (i.e., for ones where m is relatively small), but Floyd-Warshall wins for dense graphs (for ones where m is close to n^2 or even larger if we allow multiple edges). A big advantage of the Floyd-Warshall algorithm is its simplicity—we essentially only have to remember the correct order of the k, i, j loops. Unfortunately, because of the rather high complexity, it cannot be used for very large graphs.

The algorithm as presented above works only for directed graphs. However, by adding just a single line, it will also find the cheapest paths in undirected graphs. Where would you add the line? Before unrevealing the answer, try to come up with a solution yourself! Use the fact that any undirected graph can be transformed in a directed graph by replacing each edge by a pair of opposite edges. *Select to see the solution:* In the first loop, add the following line of code `delta[t][s] = min(delta[t][s], c)` to add each edge in both directions. Alternatively, replace each (undirected) edge in the input edge list by two opposite directed edges of the same cost. That's it!

As a final note, please observe the difference between the *delta* matrix computed by Floyd-Warshall and the adjacency matrix of a graph. Whereas the latter has 0/1 entries to indicate whether there is an edge between two nodes, the former has the cost of a cheapest path going from one node to the other. Interestingly, with a small modification of the Floyd-Warshall algorithm, we can reconstruct an actual cheapest path for each given pair in linear time; you find a task about it on the exercise sheet!

Minimum Spanning Tree Problem

Suppose we have a weighted undirected graph that is *connected*, i.e., a path from every vertex to every other vertex exists. We want to choose a subset of edges such that:

- spanning: the graph is still connected,
- minimum: the total cost of the chosen edges is as small as possible.

The resulting graph will have no cycles (i.e., it will be a *tree*)—if it had a cycle, it would be possible to remove one of the edges from the cycle, and the graph would be still connected.

Applications of MST

If the original graph contains the costs of connecting vertices with potential new roads, the MST can be used to find a cheapest road network which allows its users to reach every considered location. Likewise, it could be used to find an optimal communication network.

It can also be applied for taxonomy—if our vertices are species (for example, species of plants), and the cost of an edge between v and w corresponds to the differences between these two species (the more similar they are, the smaller the edge cost is), the constructed MST (especially when constructed with Kruskal's algorithm that we examine below) enables us to categorize our species into a taxonomical classification. For instance, in the context of evolution theory, we could conjecture that the evolutionary tree for our group of species is similar to the constructed tree. This method also has applications in data science—vertices are our observations, and edge costs correspond to differences between them; since Kruskal's algorithm will group similar observations together, it can be used as a simple clustering algorithm.

Kruskal's Algorithm

For simplicity, assume that every edge has a different cost.

Consider the cheapest edge e in the whole graph. It is easy to see that we can safely choose this edge. Indeed, a spanning tree T which does not include this edge cannot be minimal—adding e yields a cycle; if we remove another edge from this cycle, we get a graph which is still connected, and its total cost is smaller than T .

We repeat this for every edge in the graph, ordered from the cheapest to the most expensive one. More precisely, for every edge in this order, we check whether its two endpoints are connected—if not, we add that edge (by a reasoning similar to the one above, we can do this safely).

Note: The algorithm described above is a *greedy* algorithm since it always takes the most promising choice. In general, greedy algorithms can be also incorrect (recall our first unsuccessful attempt to solve the Knapsack problem in Lecture 6 via a greedy approach). Proving the correctness of a correct greedy algorithm is often difficult. However, in the case of Kruskal's algorithm, it is easy to prove that it is correct.

For our algorithm to work fast, we only need to find a way to quickly tell whether two vertices v and w are connected. The *disjoint set data structure*, also called *find-union data structure*, is a data structure which solves this problem efficiently. It has three operations:

- $\text{Init}(n)$: initialize n disjoint sets: $\{0\}, \{1\}, \dots, \{n-1\}$
- $\text{Union}(i,j)$: combine the set containing i and the set containing j into one set. For example, after $\text{Init}(5)$ and $\text{Union}(1,2)$ we have the following sets: $\{0\}, \{1,2\}, \{3\}, \{4\}$. After $\text{Union}(2,3)$ we have three sets: $\{0\}, \{1,2,3\}, \{4\}$.
- $\text{Find}(i)$: return an identifier of the set containing i , i.e., $\text{Find}(i)$ and $\text{Find}(j)$ will return the same value iff i and j are in the same set.

We can use a find-union structure in Kruskal's algorithm in the following way:

```
# We create n sets, one for each vertex.
fu = init(n)

# mst will contain the edges of our minimum spanning tree
mst = []

# We consider the edges one by one in the order of increasing costs
for (c,s,t) in sorted((c,s,t) for (s,t,c) in edges):
    # If the endpoints of the edge belong to two different sets ...
    if find(fu,s) != find(fu,t):
        # ...we add the the egde to mst...
        mst.append((s,t,c))
        # ...and merge the two sets together.
        union(fu,s,t)
```

What is the memory complexity? It depends on the size of the find-union structure. As we will see in the next section, the structure consists of an array of size n , hence, the memory is therefore $\Theta(n)$.

And what is the running time? It depends on the time to initially sort the edges as well as on the time to perform the find-union operations. In the next section, we will see that $\text{Init}(n)$ takes $\Theta(n)$ time and $\text{Find}()$ and $\text{Union}()$ take amortized $\Theta(\alpha(n))$ time, where α is an extremely slowly growing function called the *inverse Ackermann function*. This function is much slower than $\log n$ and, in practical applications, $\alpha(n)$ can be treated like a

constant. Recall that an amortized time of $O(\alpha(n))$ means that the total running time of a sequence of k operations is guaranteed to be bounded by $O(k\alpha(n))$ though single calls of the operations may exceed the bound of $O(\alpha(n))$. In our case, we perform $2m$ Find() and up to m Union() operations. Thus, the total running time of Kruskal's algorithm is

$$T(\text{sort edges}) + \Theta(m\alpha(n)).$$

In general, the time to sort is $\Theta(m \log m)$. We can simplify the sorting time even to $\Theta(m \log n)$ as (i) we know that the input graph is connected (otherwise there is no solution) and therefore $m \geq n-1$, and (ii) since we have $m \leq n^2$ (assuming no parallel edges) and since $\log(n^2) = \Theta(\log n)$. Observe that $\Theta(m \log n)$ dominates $\Theta(m\alpha(n))$ as $\alpha(n)$ grows much slower than $\log n$. Thus, in general, the total running time of Kruskal's algorithm is

$$\Theta(m \log n).$$

However, in the case that the edges are already sorted, or that they can be sorted with an efficient algorithm such as counting sort (for instance if all the edge costs belong to a small range of different values), the time complexity drops down to

$$\Theta(m\alpha(n))$$

which is indistinguishable from $\Theta(m)$ in practical applications.

The remaining part of this chapter is not mandatory for the course.

Find-Union Structure: Implementation

The find-union structure is implemented as an array fu of length n . As noted above, it maintains a partition of the elements $0, \dots, n-1$, i.e., a family of disjoint sets. Every set has a representative—one of its elements; the Find(i) operation will always return the representative of the set containing i . The idea is that $fu[i]$ tells us where to look when looking for the representative of i ; we could visualize this information as an arrow which points from i to $fu[i]$. If $fu[i]$ equals i , then i itself is the representative; otherwise, recursively look for the representative of $fu[i]$ (which is also the representative of i) by checking whether $fu[i]$ equals $fu[fu[i]]$ and so on; see the code below. The Union(i, j) operation works by taking the representatives of i and j , and by adding a new arrow between them; see the code.

```
def init(n):
    return [i for i in range(n)]

def find(fu, i):
    if fu[i] == i:
        return i
    else:
        return find(fu, fu[i])

def union(fu, i, j):
    fu[find(fu, i)] = find(fu, j)
```

This implementation is not very efficient—it is quite easy to find a sequence of Find operations which yields the following structure: $0 \leftarrow 1 \leftarrow 2 \leftarrow 3 \leftarrow 4 \leftarrow \dots \leftarrow n-1$ (i.e., $fu[0]$ equals 0, and, for $i > 0$, $fu[i]$ equals $i-1$). In such a case, $\text{Find}(k)$ runs in time $\Theta(k)$ —since Kruskal's algorithm calls the Find operation $\Theta(m)$ times, this would lead to a total running time of $\Omega(nm)$. However, there are two optimizations to the idea above:

- **Union by rank.** The problem with the solution above was that, when doing $\text{Union}(i, j)$, we always created an arrow from $\text{Find}(i)$ to $\text{Find}(j)$ —it was possible to create a very long path this way. However, we can do this in a better way: create an arrow from the set which is currently smaller, to the set which is currently larger. The worst case is when we always create an arrow between two sets of the same size—this way, we can only get a sequence of arrows of length k if the size of the set is at least 2^k . Consequently, the sequence of arrows is never longer than $\log_2 n$ as no set contains more than n elements.
- **Path compression.** In the bad example above, doing $\text{Find}(n-1)$ takes $\Theta(n)$ time every time we do it. But we could fix this in the easy way—when we do $\text{Find}(i)$, we change $fu[i]$ such that it directly points to the result (i.e., we compress the path starting from i to a single edge). Note that, because we are calling Find recursively for all the vertices on the way, this will also compress the path for all the edges on the way. It is possible that some individual Find operations are slow, but they make the following ones much faster—so we can hope that the amortized time complexity will be good.

When we do only *Union by rank*, the time complexity of each Find/Union operation drops to $O(\log n)$. When we do only *Path compression*, the *amortized* time complexity drops to $O(\log n)$. When we do both, the amortized time complexity drops to $\Theta(\alpha(n))$, where $\alpha(n)$ is the inverse Ackermann function. This is a very slowly growing function, which can be considered to be a constant for every practical purpose. The *iterated logarithm of x* , called $\log^* x$, is the number of (binary) logarithms necessary to bring x down to a value of at most 1. For example, for 65536 we have $65536 = 2^{16} \rightarrow 16 = 2^4 \rightarrow 4 = 2^2 \rightarrow 2 = 2^1 \rightarrow 1$, so $\log \log \log 65536 = 2 > 1$ and $\log \log \log \log 65536 = 1$. Thus, $\log^* 65536 = 4$. Now, consider the extremely large number 265536 which for sure is larger than any n used in practice. Note that we have only $\log^* 265536 = 5$. Thus, we can assume

that $\log^* n \leq 5$ for any practical value of n . We could also consider the doubly iterated logarithm (by iterating $\log^*$ in the same way), the triply iterated logarithm, etc. The inverse Ackermann function grows slower than any iterated logarithm— $\alpha(n)$ is roughly the smallest k such that the k -iterated logarithm of n is not greater than k .

Therefore, Kruskal's algorithm works in time $T(\text{sort edges}) + \Theta(m\alpha(n))$. In literature, the running time is also often given as $T(\text{sort edges}) + O(m\log^* n)$.

Ostatnia modyfikacja: wtorek, 2 czerwca 2026, 22:22

[◀ Lecture 13 - recording](#)

Przejdź do...

[Lecture 14 ▶](#)

✉ Skontaktuj się z pomocą techniczną

Jesteś zalogowany(a) jako Ondřej Marvan (Wyloguj)

2400-DS1AL#2025L#277553